

3. 🟡 The x86 Microprocessors(Resumen Traducido)

📖 Capitulo 0 — Basics Of Computers Systems

📖 0.1 — Una breve historia de los microprocesadores

🟡 Introducción

Un **microprocesador** se define como un procesador integrado en un único microchip (donde «micro» hace referencia a su tamaño reducido). Se trata de un dispositivo con capacidad de computación o procesamiento de datos. A lo largo de las décadas, tanto la potencia de cálculo como la velocidad de estos chips han aumentado vertiginosamente para dar respuesta a nuevas aplicaciones.

🟡 Línea de tiempo: La evolución de los microprocesadores de Intel

- **Década de 1950:**

Surge tempranamente la idea teórica de una computadora en un solo chip. Sin embargo, en esta época la electrónica y la tecnología de circuitos integrados estaban aún en una fase muy incipiente.

- **Finales de la década de 1960:**

La tecnología de circuitos integrados madura e incursiona firmemente en el mercado. Diversas empresas comienzan la carrera por fabricar un procesador en un solo chip; entre ellas destaca **Intel**.

- **Noviembre de 1971 — Intel 4004:**

Intel lanza el primer chip de uso general de 4 bits. Contaba con una velocidad de reloj de 108 kHz, 2.300 transistores y puertos para memoria ROM, RAM y operaciones de entrada/salida (I/O). Marcó el punto de partida de la microcomputación.

- **Abril de 1972 — Intel 8008:**

Llega la versión de 8 bits del 4004. Aunque representó un paso evolutivo, no ofrecía prestaciones espectaculares, por lo que su impacto comercial y técnico en el sector fue moderado.

- **1974 — Intel 8080 (El gran salto):**

Intel revoluciona la industria al presentar el 8080, considerado **el primer microprocesador verdaderamente utilizable y moderno**. Mantuvo el conjunto de instrucciones del 8008, pero incorporó mejoras determinantes:

- Bus de direcciones de 16 bits y bus de datos de 8 bits.

- Puntero de pila (*stack pointer*) de 16 bits en memoria externa (reemplazando la limitada pila interna de 8 niveles del 8008).
- Contador de programa de 16 bits.
- 256 puertos de I/O independientes del espacio de direccionamiento.
- Un pin de señal que permitía ubicar la pila en un banco de memoria separado.
- **1976 — Intel 8085:**

Intel actualiza el diseño del 8080 creando un procesador más simple y bien estructurado. Aunque nunca se implementó en una PC comercial, alcanzó gran popularidad y se siguió utilizando ampliamente en sistemas embebidos.

- **1978 — Intel 8086 (Nacimiento de la arquitectura x86):**

Intel introduce su primer procesador completo de 16 bits, marcando el origen de la célebre arquitectura **x86**.

- **1979 — Intel 8088:**

Lanzamiento de una variante del 8086 con arquitectura interna de 16 bits, pero con un bus de datos externo reducido a 8 bits para abaratar costes de compatibilidad con componentes ya existentes en el mercado.

Otros actores clave del mercado (Competencia)

A la par de Intel, otros fabricantes realizaron aportes significativos:

- **Motorola (posteriormente Freescale):** Desarrolló diseños originales propios a partir de su chip **6800** y versiones posteriores. En términos de capacidad computacional, sus procesadores igualaban e incluso superaban a los de Intel, orientándose con el tiempo a nichos de mercado distintos y separados.
- **AMD:** En lugar de crear diseños desde cero, comenzó trabajando en la mejora del diseño del 8080 de Intel mediante acuerdos de licencia.
- **Zilog Corporation:** Creó el célebre **Z-80**, un procesador de 8 bits compatible a nivel binario con el 8080 de Intel y con prestaciones superiores. Aunque la compañía tuvo dificultades para competir a escala frente a gigantes comerciales, el Z-80 logró un éxito masivo que perdura en aplicaciones embebidas.

La consagración de la familia x86 en la PC

El punto de inflexión para Intel ocurrió en **1981**, cuando IBM seleccionó el microprocesador **8088** para su computadora personal (PC). La elección del 8088 sobre el 8086 se debió a razones de coste y compatibilidad: aunque ambos compartían la misma arquitectura interna de 16 bits, el 8088 contaba con un bus de datos externo de 8 bits, facilitando su conexión con los periféricos económicos de la época.

A partir de este hito, la evolución de la arquitectura x86 continuó de forma lineal:

- **80186 (1982):** Nunca se utilizó en computadoras personales; se orientó al mercado de sistemas embebidos.
- **80286 (1982):** Se integró en las PC con arquitectura y bus externo de 16 bits. Introdujo innovaciones clave como la **memoria virtual** y las operaciones en **modo protegido**.
- **80386 (1985):** Marcó el gran salto a los 32 bits, tanto a nivel interno como en su bus externo.
- **Generaciones posteriores (80486 a Core-2 Quad):** La arquitectura interna se consolidó en los 32 bits, mientras que la línea **Pentium** y sus sucesores aumentaron el ancho del bus de datos externo a 64 bits para elevar el rendimiento.

La filosofía x86 y el estándar de la industria

El término **x86** define el conjunto de instrucciones en lenguaje máquina introducido originalmente con el procesador Intel 8086. Aunque fue un diseño propio de Intel, otros fabricantes como AMD adoptaron este mismo «vocabulario» técnico, manteniendo compatibilidad pese a implementar diseños internos distintos.

- **Evolución y retrocompatibilidad:** Con el paso de los modelos (80186, 80286, 80386, 80486 y la línea Pentium), el conjunto de instrucciones x86 se fue expandiendo sin perder la compatibilidad hacia atrás: cualquier software escrito para un 8086 puede ejecutarse en un Pentium, aunque no a la inversa.
- **Dominio del mercado:** La arquitectura x86 se consolidó como el estándar *de facto* en toda la gama de computación personal y profesional, abarcando desde computadoras de escritorio y portátiles hasta servidores y supercomputadoras, relegando a los competidores alternativos a cuotas de mercado reducidas.

0.2 — Bases de la Arquitectura de Computadoras

Originalmente "computar" se refería solo a cálculos aritméticos, pero hoy en día el término se asocia más con el "procesamiento de datos", ya que las computadoras manejan grandes volúmenes de información.

Una computadora está compuesta por dos grandes categorías:

- **Hardware:** los componentes físicos de la máquina.
- **Software:** el conjunto de programas que le indican al hardware qué tareas realizar.

En cuanto al hardware, el sistema se organiza en tres partes principales:

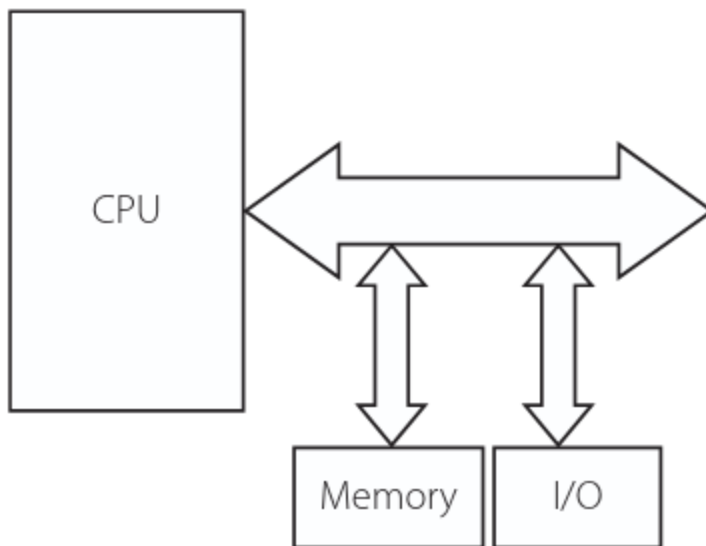
1. **CPU (Unidad Central de Procesamiento)**
2. **Memoria**

3. Dispositivos de entrada/salida

La CPU es considerada el "corazón" de la computadora, ya que es la que le da "vida" al sistema. Generalmente es un microprocesador (un chip autónomo) y su función es procesar los datos según las instrucciones de los programas. Estas instrucciones son decodificadas por la CPU, la cual genera las señales de control necesarias para activar sus dos componentes internos clave:

- La **unidad aritmético-lógica** (encargada de los cálculos)
- La **unidad de control** (encargada de coordinar las operaciones)

Todo este funcionamiento está sincronizado por una **señal de reloj**, un tren de pulsos de frecuencia fija que coordina tanto la actividad interna de la CPU como su comunicación con el bus del sistema.



Buses del Sistema

Un **bus** es un conjunto de cables que conecta los componentes de la computadora (CPU, memoria y E/S). La CPU se comunica con la memoria o con los dispositivos de E/S a través del mismo bus, pero solo con uno a la vez, por lo que puede haber conflictos si ambos necesitan usarlo simultáneamente. El bus del sistema se divide en tres tipos:

1. Bus de datos

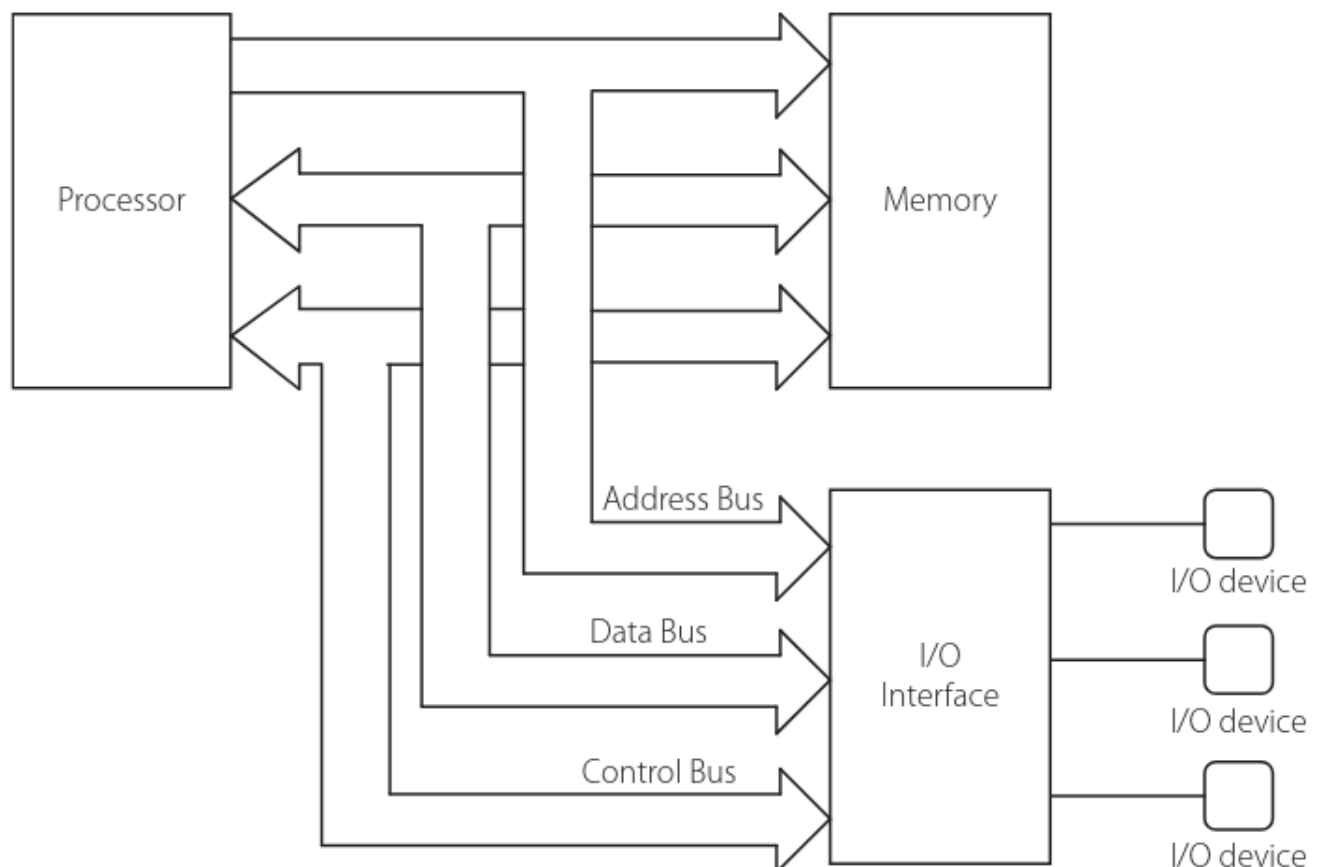
- Transporta los datos y es **bidireccional** (la CPU envía y recibe datos).
- Su ancho determina la velocidad de transferencia, el tamaño de los registros internos y la capacidad de procesamiento de la CPU.
- Ejemplo: el 8086 tiene un bus de datos de 16 bits, mientras que el 80486 tiene 32 bits (por lo que puede procesar el doble de datos a la vez).

2. Bus de direcciones

- Determina la cantidad máxima de memoria que la CPU puede direccionar.
- Es **unidireccional**, ya que solo la CPU envía direcciones (tanto para memoria como para dispositivos de E/S).
- Ejemplo: el 8086 con 20 bits puede direccionar 2^{20} ubicaciones (1 MB), mientras que el Pentium con 32 bits puede direccionar 2^{32} ubicaciones (4 GB).

3. Bus de control

- Contiene las señales que controlan las acciones sobre el bus, como lectura/escritura en memoria o E/S, interrupciones y acceso directo a memoria (DMA).
- Incluye señales como *Memory Read*, *I/O Read*, *Memory Write* e *Interrupt Acknowledge*.
- A diferencia de los otros buses, las señales de control pueden viajar **en ambas direcciones** (algunas van de la CPU hacia los dispositivos, y otras, como las interrupciones, van de los dispositivos hacia la CPU).



Que significa el termino "Bus Contention"

En el contexto de un sistema de computadoras, **bus contention** (contención de bus o conflicto de bus) ocurre cuando **dos o más dispositivos intentan transmitir datos a través del mismo bus compartido al mismo tiempo**.

Dado que un bus físico es una colección de cables donde las señales se representan mediante voltajes eléctricos, si dos componentes intentan enviar datos simultáneamente (por ejemplo, uno intenta poner un 1 enviando alto voltaje y otro un 0 enviando bajo voltaje), las señales se mezclan, se corrompen y pueden provocar un cortocircuito a nivel eléctrico.

🚫 Puntos clave sobre la contención de bus:

- **¿Cuándo ocurre?** Típicamente cuando la CPU y un controlador de **DMA (Direct Memory Access)** o un dispositivo de **I/O** necesitan acceder a la memoria principal a través del bus del sistema al mismo instante.
- **¿Cómo se resuelve?** Mediante un mecanismo llamado **Arbitraje de Bus (Bus Arbitration)**. Un circuito de hardware (el árbitro de bus) decide qué dispositivo obtiene el control (*Bus Master*) según un esquema de prioridades y hace esperar al resto mediante estados de espera (*Wait States*).
- **Efecto directo:** Genera cuellos de botella y reduce el rendimiento general del sistema mientras los componentes esperan a que el bus quede libre.

Ejemplo Practico

Escenario: Presionas 2 + 3 en el teclado y ves el resultado 5 en pantalla

Fase 1: Entrada y Almacenamiento (I/O Memoria CPU)

- **Entrada (I/O):** Presionas las teclas. El dispositivo de I/O genera una **interrupción** que viaja por el **Bus de Control** (de I/O a la CPU).
- **Bus de Datos (I/O CPU):** Los valores 2 y 3 viajan desde el teclado (I/O) hacia la CPU.
- **CPU:** Recibe la instrucción y usa el **Bus de Direcciones** (unidireccional) para apuntar a las celdas 0x01 y 0x02 de la memoria.
- **Bus de Datos (CPU Memoria):** La CPU envía 2 y 3 a la **Memoria Principal** para guardarlos temporalmente. (*Primera dirección del bus de datos*).

Fase 2: Procesamiento (Memoria CPU)

- **Bus de Control:** La CPU envía la señal **Memory Read** a la Memoria.
- **Bus de Direcciones:** La CPU solicita las direcciones 0x01 y 0x02.
- **Bus de Datos (Memoria CPU):** La Memoria devuelve los números 2 y 3 a la CPU. (*Segunda dirección del bus de datos*).
- **CPU:** Realiza internamente la suma (2 + 3) y genera el resultado 5.

Fase 3: Salida (CPU I/O)

9. **Bus de Control:** La CPU activa la señal **I/O Write** hacia el puerto de salida.

10. **Bus de Direcciones:** La CPU apunta al puerto del monitor (I/O).
11. **Bus de Datos (CPU I/O):** El valor 5 viaja desde la CPU hacia el monitor (I/O) para mostrarse en pantalla.

El Procesador

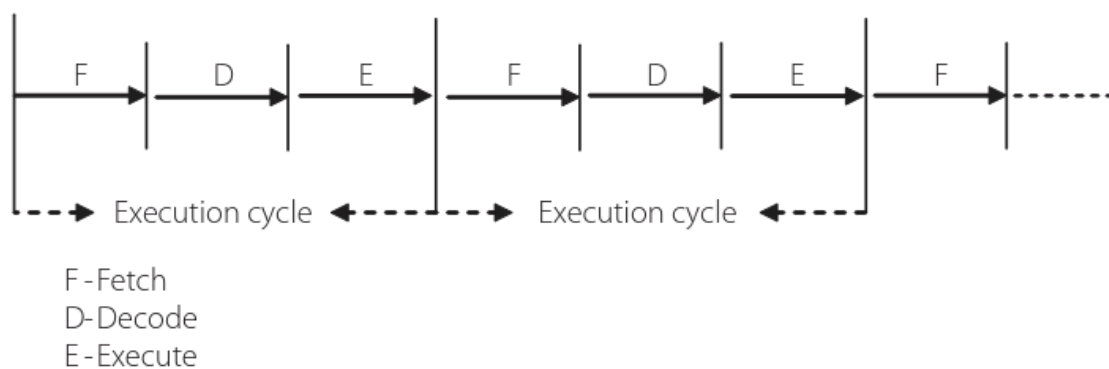
El procesador (o microprocesador) es el componente encargado de controlar toda la actividad del sistema. Para esto, realiza continuamente un ciclo de tres pasos, llamado **ciclo de ejecución**:

1. **Fetch (búsqueda):** trae la instrucción desde la memoria.
2. **Decode (decodificación):** interpreta qué acción debe realizar esa instrucción.
3. **Execute (ejecución):** realiza la acción indicada.

Para llevar a cabo este ciclo, el procesador cuenta con:

- **Unidad de control**, que se encarga de buscar y decodificar las instrucciones.
- La **ALU** (unidad aritmético-lógica), que ejecuta las operaciones aritméticas/lógicas requeridas.

Este ciclo fetch-decode-execute se repite de manera continua e infinita. Una consecuencia importante es que, normalmente, la ejecución de instrucciones es **secuencial**: una instrucción se completa antes de pasar a la siguiente. Sin embargo, esta secuencia puede romperse cuando aparece una **instrucción de salto o "branch"**, que hace que el procesador continúe la ejecución desde una nueva ubicación de memoria, en lugar de seguir con la siguiente instrucción en orden.



El Reloj

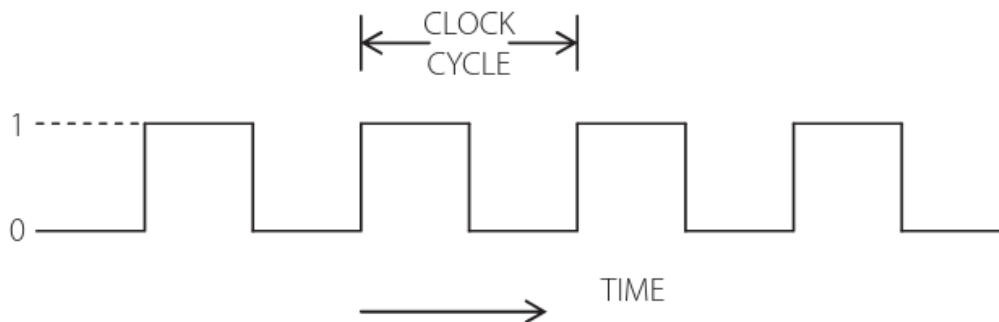
Todas las actividades del procesador y de los buses están sincronizadas por un **reloj**, que consiste en una onda cuadrada de una frecuencia determinada.

- El **período del reloj (T)**, también llamado tiempo de ciclo, es el recíproco de la frecuencia:
 $T = 1/f$.

- Un **ciclo de ejecución** puede requerir varios períodos de reloj (varios "clock cycles"), y esto depende de:
 - Las características arquitectónicas del procesador.
 - La complejidad de la instrucción que se ejecuta.
 - La cantidad de ciclos necesarios para acceder a la memoria (ya que el ciclo de ejecución también implica traer instrucciones y datos de memoria).

En términos generales, una **mayor velocidad de reloj implica un procesamiento más rápido**: por ejemplo, un reloj de 3 GHz permite procesar más rápido que uno de 1 GHz.

Sin embargo, hay una limitación importante: la **tecnología del procesador debe ser capaz de soportar** la frecuencia de reloj utilizada. No alcanza con "subir" la frecuencia arbitrariamente; el diseño físico del chip debe permitir operar a esa velocidad.



Memoria Principal

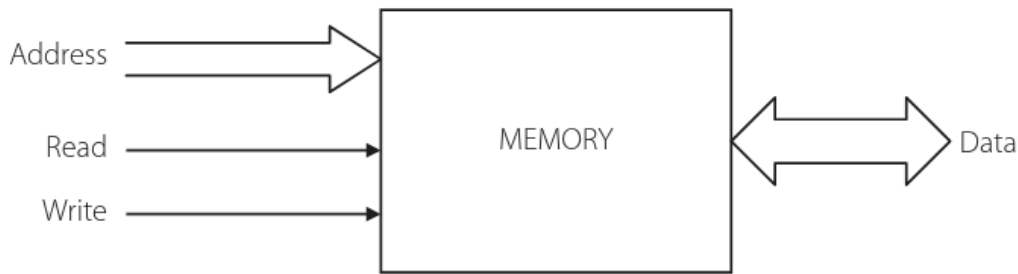
La memoria de un sistema de cómputo incluye tanto la **memoria primaria** como la **memoria secundaria**, aunque por ahora el texto se centra únicamente en la **memoria primaria** (o memoria principal), que suele ser **RAM** (Random Access Memory).

- La memoria se organiza en **bytes**, y la capacidad de un chip de memoria se expresa en la cantidad de bytes que puede almacenar (por ejemplo: 256 bytes, 1 KB, 1 MB, etc.).
- Si una computadora tiene un espacio total de memoria de, por ejemplo, 20 MB, puede lograrlo combinando chips RAM de distintas capacidades disponibles.

Existen **dos operaciones básicas** asociadas a la memoria:

1. **Lectura (read)**: transfiere el dato almacenado en una posición de memoria hacia la CPU, **sin borrar** el contenido original en memoria.
2. **Escritura (write)**: coloca un nuevo dato en una posición de memoria, **sobrescribiendo** el valor anterior.

Ambas operaciones requieren cierto tiempo para completarse, lo cual se conoce como **tiempo de acceso (access time)**.



Ciclo de lectura de memoria (Memory Read Cycle)

Pasos típicos:

1. El procesador coloca en el **bus de direcciones** la dirección de la posición de memoria a leer.
2. Se activa (assert) la señal de **memory read**, parte del bus de control.
3. Se espera hasta que el contenido de esa dirección aparezca en el **bus de datos**.
4. Se transfiere ese dato del bus de datos hacia el procesador.
5. Se desactiva la señal de memory read, finalizando la operación (la dirección en el bus ya no es relevante).

Ciclo de escritura de memoria (Memory Write Cycle)

Pasos típicos:

1. Se coloca en el **bus de direcciones** la dirección donde se va a escribir.
2. Se coloca en el **bus de datos** el dato a escribir.
3. Se activa la señal de **memory write**, parte del bus de control.
4. Se espera hasta que el dato quede almacenado en la posición indicada.
5. Se desactiva la señal de memory write, finalizando la operación.

Punto clave: ambas operaciones están **sincronizadas con el reloj del sistema**. Por ejemplo, un procesador 8086 necesita como mínimo **cuatro ciclos de reloj** para completar una lectura o escritura. Estos cuatro ciclos conforman el "memory read cycle" o "memory write cycle" del procesador. Otros procesadores pueden requerir más o menos ciclos para las mismas operaciones.

Sistema de I/O

Para que una computadora se comuniqué con el mundo exterior, necesita **periféricos**. Estos pueden ser:

- De **entrada** exclusivamente (teclado, mouse).

- De **salida** exclusivamente (impresora, monitor).
- De **entrada y salida** (módem).

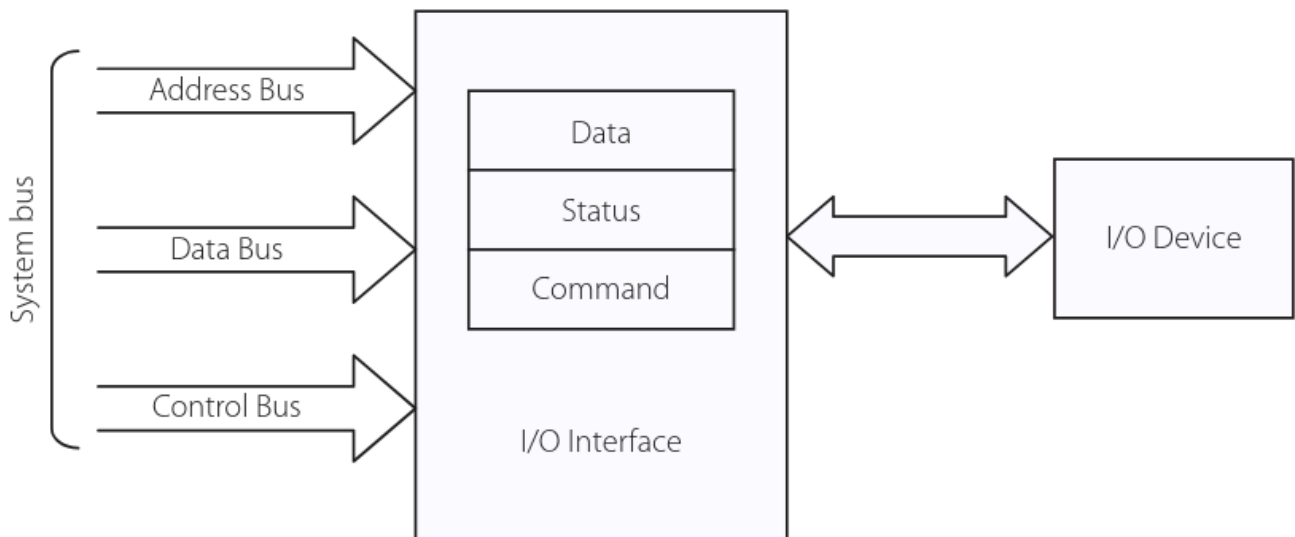
El problema: el procesador no puede comunicarse directamente con los periféricos, porque cada uno tiene características distintas (voltajes, velocidades, estándares) que el procesador no entiende, y este no cuenta con las señales de control necesarias para manejar esa diversidad.

La solución: cada periférico utiliza un **controlador** (a menudo un chip especializado) que actúa como interfaz entre el dispositivo y el procesador. Este controlador:

- Entiende las características específicas del periférico.
- Genera las señales de control adecuadas para que el procesador pueda comunicarse con él.

Ejemplos de estos controladores: el chip de interfaz de teclado/pantalla, el chip de puerto paralelo, el chip de comunicación serial, entre otros. Todos estos chips son **programables**, ya que cuentan con registros de comandos, datos y estado, lo que permite programarlos para lograr una comunicación correcta entre el procesador y cualquier periférico.

Conclusión del capítulo: una computadora puede entenderse, en definitiva, como un conjunto de componentes —memoria y dispositivos de E/S de distintos tipos, aplicaciones y especificaciones— que trabajan en conjunto.



0.3 — Lenguaje Maquina

 **Lenguaje Maquina, Lenguaje Ensamblador y lenguajes de alto nivel**

Para que una computadora pueda sernos útil, necesitamos programarla utilizando un lenguaje que ella pueda entender. Existen tres niveles de lenguaje de programación:

- **Lenguaje de máquina (Machine Language):** es el lenguaje nativo de la computadora, compuesto por unos y ceros. Cada secuencia binaria constituye un código de operación u "opcode", que le indica al procesador qué acción debe realizar, existiendo un código distinto para cada operación (por ejemplo, uno para sumar y otro para restar). El problema de este lenguaje es que resulta muy engorroso y propenso a errores para los seres humanos, ya que no somos buenos recordando o manejando códigos binarios.
- **Lenguaje ensamblador (Assembly Language):** surge para facilitar la comunicación con la computadora, siendo un nivel más comprensible para las personas. En lugar de códigos binarios, utiliza "mnemónicos", que son símbolos que representan directamente cada código de máquina; por ejemplo, la suma se representa con el símbolo "ADD" y la multiplicación con "MUL". De esta manera, el usuario solo necesita recordar estos símbolos y su sintaxis, en vez de código binario. Sin embargo, como la computadora no entiende estos mnemónicos, es necesario un programa llamado **ensamblador**, que se encarga de traducir el lenguaje ensamblador a lenguaje de máquina. Esta traducción es de tipo uno a uno, es decir, cada símbolo se corresponde con un único código de máquina. Además, como cada procesador tiene su propio lenguaje de máquina, también tiene su propio lenguaje ensamblador, por lo que no son universales entre distintos procesadores.
- **Lenguajes de alto nivel (High Level Language):** como C, FORTRAN o COBOL, cuyo vocabulario y gramática se asemejan al lenguaje humano, lo que los hace mucho más fáciles de entender y escribir. Una ventaja clave es que no dependen de un procesador específico: un mismo programa puede ejecutarse en distintos procesadores, siempre que exista un **compilador** para ese lenguaje, es decir, un software encargado de traducir las instrucciones del lenguaje de alto nivel a un nivel inferior (ensamblador o máquina), ya que en última instancia el procesador siempre necesita trabajar con código de máquina.

Por último, el texto que escribimos en ensamblador o en un lenguaje de alto nivel se denomina **código fuente**, mientras que el resultado de su traducción por parte del compilador o ensamblador se llama **código objeto**, el cual es ejecutable porque el procesador puede interpretarlo y llevar a cabo las tareas que indica.

Lenguaje Ensamblador VS Lenguaje de Alto nivel

Criterio	Lenguaje Ensamblador	Lenguaje de Alto Nivel
Dependencia del procesador	Específico de cada procesador (ej: código de 8086 no sirve para 8085)	Independiente del procesador, siempre que exista un compilador
Curva de aprendizaje	Requiere conocer a fondo la arquitectura (registros, flags,	Fácil de aprender, sintaxis cercana al lenguaje humano

Criterio	Lenguaje Ensamblador	Lenguaje de Alto Nivel
	manejo de datos)	
Eficiencia y velocidad	Código compacto y muy rápido (hasta 100x más veloz)	Código menos compacto; una instrucción puede generar decenas o cientos de instrucciones máquina
Control sobre el hardware	Acceso directo al hardware; ideal para kernels, drivers y control de máquina	Abstracto, sin acceso directo al hardware
Costos de desarrollo	Requiere programadores especializados; mayor tiempo de desarrollo	Permite programadores menos especializados; menor tiempo de desarrollo y mantenimiento
Prioridad principal	Velocidad y control	Portabilidad y facilidad de desarrollo

0.4 — Arquitectura RISC y CISC

Dos términos frecuentes en arquitectura de computadoras son **RISC** (Reduced Instruction Set Computer) y **CISC** (Complex Instruction Set Computer).

Contexto histórico

En los primeros tiempos del desarrollo de microprocesadores, la tendencia era implementar instrucciones complejas totalmente en hardware, dando origen a la filosofía CISC. La arquitectura **x86** se basó en este enfoque desde el inicio. Para cuando la filosofía RISC se popularizó y maduró como alternativa viable, los procesadores CISC x86 ya se habían consolidado en el mercado, por lo que muchos desarrolladores prefirieron no arriesgarse a migrar a un dominio poco probado. Sin embargo, la mayoría de los procesadores más nuevos adoptaron la filosofía RISC —como **ARM, PowerPC y Sparc (de Sun)**—, muchos de ellos utilizados en sistemas embebidos.

Definición y características de CISC

CISC se basa en implementar instrucciones complejas totalmente en hardware (por ejemplo, la instrucción de multiplicación requiere un multiplicador dedicado). Como el hardware es rápido, la ejecución es rápida, pero tener muchas instrucciones complejas implementadas de esta manera eleva considerablemente el "presupuesto de hardware" (la cantidad y complejidad de circuitería necesaria).

Definición y características de RISC

RISC parte de la premisa de que, en promedio, las instrucciones complejas se usan relativamente poco. Por eso, en lugar de implementarlas directamente en hardware, se

construyen combinando instrucciones simples. Esto reduce el presupuesto de hardware y mantiene un set de instrucciones pequeño, con instrucciones simples ejecutadas en un solo ciclo de reloj, aunque a cambio requiere más trabajo de software para lograr las mismas funciones complejas (en esencia, "cambiar software por hardware"). Existe una ironía en esto: muchos procesadores RISC tienen tantas instrucciones complejas como los CISC, lo cual se explica porque dichas instrucciones se implementan mediante **microprogramación** (un método para construir la unidad de control descomponiendo instrucciones en una secuencia de pequeños pasos programados), en lugar de una implementación directa en hardware.

Situación actual

La controversia RISC vs. CISC sigue vigente, pero la diferencia práctica entre ambos enfoques se ha vuelto casi imperceptible, quedando reducida a un nivel más filosófico que técnico. Un ejemplo claro es Intel, que sostuvo la filosofía CISC durante años, pero terminó cediendo terreno a RISC al diseñar el **Pentium Pro**, cuyas instrucciones complejas se descomponen internamente en instrucciones simples tipo RISC. Por eso se dice que el Pentium Pro es, en el fondo, "un procesador RISC que ejecuta instrucciones CISC".

 0.8 — Unidades de capacidad de memoria

Un dispositivo de memoria almacena datos, y su capacidad se especifica en **múltiplos de bytes**, ya que la memoria está organizada en bytes (cada byte ocupa una posición de memoria). Por ejemplo, un dispositivo con 100 posiciones almacena 100 bytes.

Conceptos base:

- Un **byte** equivale a 8 bits.
- Un **word** (palabra) no tiene un tamaño fijo definido: depende del procesador. Por ejemplo, en el 8086 un word es de 16 bits, mientras que en un procesador de 32 bits, el word puede ser de 32 bits.
- La capacidad de memoria siempre se expresa en **bytes**.

Unidades de capacidad de memoria (todas basadas en potencias de 2):

Unidad	Equivalencia	Valor en bytes
—	2 ⁸	256 bytes
1 KiloByte (KB)	2 ¹⁰	1.024 bytes
64 KB	2 ¹⁶	65.536 bytes
1 MegaByte (MB)	2 ²⁰	1.048.576 bytes
1 GigaByte (GB)	2 ³⁰	1.073.741.824 bytes
1 TeraByte (TB)	2 ⁴⁰	1.099.511.627.776 bytes

Unidades superiores (poco comunes por ahora, pero cada vez más relevantes):

- **PetaByte (PB)** = 2^{50} bytes
- **ExaByte (EB)** = 2^{60} bytes
- **ZettaByte (ZB)** = 2^{70} bytes

Idea clave: cada unidad superior equivale a multiplicar la anterior por 1.024 (2^{10}), ya que el sistema binario usa potencias de 2 en lugar de potencias de 10 como el sistema decimal.

Capítulo 1 — La arquitectura del 8086

En este capítulo tendrán que quedar claros los conceptos de:

- La arquitectura interna del 8086
- Los registros del 8086 y sus funciones específicas
- Las operaciones de banderas del 8086
- La técnica de segmentación de memoria usada por la familia x86
- Como realizar cálculos de direcciones de memoria
- Los diferentes modos de direccionamiento del 8086.

1.1 La Arquitectura interna del 8086

El procesador 8086 es de 16 bits tanto interna como externamente: su bus de datos y sus registros internos manejan 16 bits, por lo que puede acceder a datos externos de a 16 bits por vez a través del bus de datos. Sin embargo, también puede trabajar con datos de 8 bits (bytes).

Para entender su arquitectura, se analiza primero el diagrama de bloques simplificado (y luego uno más detallado). Aunque no se muestra explícitamente en el diagrama, todas las acciones del procesador están sincronizadas por un reloj de sistema, que provee la temporización básica. Además, existe una unidad de control que genera las señales de control necesarias para el funcionamiento general del procesador.

El diagrama interno se divide en dos unidades lógicas principales:

- **BIU (Bus Interface Unit / Unidad de Interfaz de Bus)**
- **EU (Execution Unit / Unidad de Ejecución)**

Estas dos unidades interactúan entre sí a través del bus interno, pero funcionan de manera separada, cada una con funciones bien definidas. El texto propone explorar la configuración interna de cada una de estas unidades por separado.

Figure 1.1b | Internal block diagram of the 8086 (detailed)

